

INTELLIGENT FRAUD TRANSACTION DETECTION AND AML CASE MANAGEMENT SYSTEM

A Project
Presented to the
University of Dhaka, Bangladesh

In Partial Fulfillment
of the Requirements for the Degree
Master in Information Technology (MIT)
or
Post Graduate Diploma in Information Technology (PGDIT)

By
<Student Name>
<Roll / Registration Number>
<Academic Year>

August 2026

SIGNATURE PAGE

PROJECT:	Intelligent Fraud Transaction Detection and AML Case Management System
AUTHOR:	<Student Name>
ROLL:	<Roll / Registration Number>
DATE SUBMITTED:	August 2026
SUPERVISED BY:	<Supervisor Name>
DESIGNATION:	<Supervisor Designation>
INSTITUTION:	University of Dhaka, Bangladesh
SUPERVISOR'S APPROVAL:	Signature and Date: _____

ACKNOWLEDGEMENTS

I express sincere gratitude to my project supervisor for continuous guidance, technical criticism, and encouragement throughout this work. I also acknowledge the faculty members and classmates whose feedback helped refine the scope from a basic anomaly detector into an auditable AML platform. Finally, I am grateful to my family for their patience and support during the design, implementation, testing, and documentation of this project.

This project benefited from open-source technologies and publicly documented machine-learning methods. Their communities made it possible to construct a complete prototype spanning transaction ingestion, feature engineering, model training, model comparison, risk policy configuration, case management, and suspicious transaction report generation.

DECLARATION

I declare that this project report is an original account of the work completed for the Intelligent Fraud Transaction Detection and AML Case Management System. Material derived from external publications and software documentation is acknowledged in the references. The system is an academic prototype and does not claim that an anomaly is legally proven fraud; suspicious outcomes require authorized human review.

Student's Name: <Student Name>

Student's Signature: _____

ABSTRACT

Financial institutions process large volumes of transactions while needing to identify unusual activity quickly, explain decisions to analysts, and preserve evidence for regulatory review. A single machine-learning model is insufficient because institutions may have no confirmed fraud labels, transaction behaviour changes as history grows, customer-specific patterns differ from peer behaviour, and model predictions alone do not represent deterministic anti-money-laundering obligations.

This project implements an Intelligent Fraud Transaction Detection and AML Case Management System using Angular, Spring Boot, FastAPI, and Microsoft SQL Server. A global setting selects unsupervised or supervised learning. Unsupervised mode uses Isolation Forest, Autoencoder, and Local Outlier Factor; supervised mode uses XGBoost, Random Forest, and Logistic Regression. Both modes feed a layered risk score combining customer behaviour, peer behaviour, the selected ML ensemble, and deterministic AML rules. Weights normalize to 100%, MEDIUM and HIGH outcomes create cases, and HIGH outcomes generate draft STR XML after transaction commit.

The main research feature evaluates how model behaviour changes when the oldest 10%, 25%, 50%, and 100% of an immutable snapshot are used. Label-free diagnostics are reported for unsupervised models, while labelled chronological evaluation is used for supervised models. Scalability is addressed through persisted point-in-time features, closed business dates, keyset export, checksummed Parquet snapshots, bounded evaluation, asynchronous case work, pagination, and versioned artifacts. The result separates anomaly evidence from confirmed fraud and supports analyst review, false-positive feedback, manual cases, and STR preparation.

Keywords: fraud detection, anomaly detection, anti-money laundering, supervised learning, unsupervised learning, model comparison, explainable risk scoring, case management, STR XML, scalable architecture

TABLE OF CONTENTS

Signature Page	i
Acknowledgements	ii
Declaration	iii
Abstract	iv
Table of Contents	v
List of Tables	vi
List of Figures	vii
Chapter 1: Introduction	1
1.1 Background	1
1.2 Motivation	1
1.3 Problem Statement	1
1.4 Objectives	2
Chapter 2: Background and Related Approaches	4
Chapter 3: System Analysis and Architecture	6
Chapter 4: Methodology	9
Chapter 5: Implementation	13
Chapter 6: Testing and Results	18
Chapter 7: Problems and Solutions	21
Chapter 8: Conclusion and Future Work	23
References	25
Appendices	26

LIST OF TABLES

Table 1. Functional requirements	6
Table 2. Non-functional requirements	6
Table 3. Dual learning model catalog	9
Table 4. Evaluation metrics by learning mode	12
Table 5. Main implementation technologies	13
Table 6. Current unsupervised model results on 50,000 rows	18
Table 7. Automated validation summary	19
Table 8. Problem-to-solution matrix	21

LIST OF FIGURES

Figure 1. Three-application architecture	7
Figure 2. Point-in-time transaction decision flow	8
Figure 3. Layered risk scoring	11
Figure 4. Governed training and comparison pipeline	12
Figure 5. Risk policy configuration interface	14
Figure 6. Transaction entry interface	15
Figure 7. Explainable transaction decision	16
Figure 8. Model comparison results	18

CHAPTER 1

INTRODUCTION

1.1 Background

Fraud monitoring and anti-money-laundering operations must identify unusual transactions without interrupting legitimate banking activity. Conventional rule engines are transparent but can be rigid and generate large numbers of false alerts. Machine-learning methods can discover complex patterns, but their raw scores are difficult to compare, may drift as the amount of history changes, and can fail silently when feature engineering differs between training and prediction.

The project began as an anomaly detector for a bank transaction dataset containing transaction identity, account, amount, date, type, location, channel, customer age, occupation, login attempts, balance, and previous transaction date. It evolved into a broader AML decision-support system that persists point-in-time features, compares models on growing chronological datasets, supports labelled and unlabelled learning, explains layered risk contributions, and manages the analyst workflow after a suspicious decision.

1.2 Motivation

A bank may store millions of transactions and cannot repeatedly send its entire lifetime history through one synchronous request. It may also lack reliable fraud labels because alerts are not equivalent to confirmed fraud. Even when a model detects an anomaly, analysts need to know whether the amount was unusual for the customer, unusual for comparable peers, associated with velocity or structuring rules, or simply outside a model boundary. These operational and scientific limitations motivated a system that separates evidence layers and preserves an audit trail.

1.3 Problem Statement

- Transaction datasets often have no trustworthy class label, preventing valid accuracy, precision, recall, and F1 claims.

- A model that looks good on a small sample may behave differently when historical data grows.
- Raw scores from different algorithms use incompatible scales and cannot be added directly.
- Scoring one transaction can become slow if the system scans large historical tables or retrains models synchronously.
- Customer behaviour, peer behaviour, AML rules, and ML evidence can disagree and require a governed method of combination.
- An anomaly alert alone is not a complete operational outcome; investigators need cases, explanations, notes, false-positive handling, and STR preparation.
- Administrative functions require authentication, authorization, model versioning, and reproducible training data.

1.4 Project Objectives

1. Build an end-to-end web platform for upload, transaction checking, training, model comparison, configuration, case review, and STR XML generation.
2. Support unsupervised and supervised learning through one system-level mode while keeping their models and evaluation metrics scientifically separate.
3. Measure how model behaviour changes using the oldest 10%, 25%, 50%, and 100% of an immutable dataset.
4. Combine transparent customer behaviour, peer behaviour, ML evidence, and deterministic AML rules into a normalized 0-100 risk score.
5. Make suspicious decisions explainable through reason codes, component scores, model diagnostics, and persisted evidence.
6. Design training, data export, and real-time decision paths that can evolve toward bank-scale workloads.

1.5 Scope

The implemented scope includes CSV and Excel ingestion, real-time transaction creation, persisted feature vectors, cold-start settings, global learning mode, six supported

model choices across two learning modes, mode-aware tuning, chronological model comparison, weighted risk policy configuration, JWT authentication, role authorization, automatic and manual cases, pagination, analyst notes, false-positive marking, and draft STR XML. The project is an academic prototype: it does not connect to a production core-banking system, submit reports to a regulator, or replace human adjudication.

1.6 Significance

The primary contribution is not a claim that one algorithm proves fraud. The contribution is an integrated framework for selecting and governing evidence: it tells users which methodology is valid for their data, how models behave as history grows, how non-model rules affect the final result, and why a case was generated. This makes the prototype useful for academic evaluation and for discussing production AML design constraints.

CHAPTER 2

BACKGROUND AND RELATED APPROACHES

2.1 Rule-Based Transaction Monitoring

Traditional monitoring uses deterministic thresholds for structuring, velocity, unusual amount, sanctions, location, and channel behaviour. Rules are easy to explain and can implement mandatory controls, but static thresholds may not reflect individual customer baselines. This project therefore retains rules as one explicit layer rather than asking ML models to rediscover every legal or operational control.

2.2 Unsupervised Anomaly Detection

When labels are unavailable, the system uses three complementary detectors. Isolation Forest isolates rare observations through randomized trees; the Autoencoder learns to reconstruct common feature patterns and treats high reconstruction error as unusual; Local Outlier Factor compares the local density around one transaction with the density around its neighbours. Their outputs indicate abnormality rather than confirmed fraud.

2.3 Supervised Fraud Classification

When an auditable binary label is available, supervised classifiers can learn the relationship between historical features and reviewed outcomes. XGBoost provides a strong nonlinear boosted-tree model, Random Forest provides a robust class-balanced ensemble, and Logistic Regression provides an interpretable probability baseline. The comparison protocol emphasizes precision-recall measures because fraud datasets are normally imbalanced.

2.4 Explainable Layered Risk

Pure voting creates unintuitive outcomes when three models agree but the ML layer owns only part of the total risk budget. The project replaces this ambiguity with an explicit layered score. Each component produces a normalized value between zero and one; configured top-level weights sum to one; selected ML model weights also sum to one

inside the ensemble. The UI shows both anomaly votes for diagnostics and the weighted score that actually determines the risk level.

2.5 Research Gap Addressed

Many demonstrations train one model on one fixed dataset and report either accuracy without proving label quality or anomaly counts without studying stability. This project adds a chronological growth experiment, separates label-free and label-based metrics, includes transparent non-ML layers, and connects evaluation to a governed operational workflow. It therefore studies both model behaviour and system behaviour.

CHAPTER 3

SYSTEM ANALYSIS AND ARCHITECTURE

3.1 Stakeholders and Use Cases

Administrators configure the learning mode, tuning values, risk policy, training runs, and model governance. Reviewers inspect cases, search transactions, add notes, mark false positives, and generate STR XML. External transaction sources upload files or call the transaction API. The ML service trains and scores models but does not own business records or case decisions.

Table 1. Functional requirements

Actor	Major functions
Administrator	Settings, uploads, training operations, model tuning, comparison, risk policy, governance
AML Reviewer	Transaction checks, case search, notes, false-positive review, manual case creation, STR XML
Transaction Source	Bulk CSV/Excel upload or real-time transaction API
ML Service	Feature alignment, model training, prediction, comparison metrics, artifact persistence

3.2 Non-Functional Requirements

Table 2. Non-functional requirements

Quality	Requirement and implementation direction
Security	Stateless JWT authentication, BCrypt password hashes, server-side role authorization, deny-by-default API policy
Explainability	Reason codes, component scores, model diagnostics, feature summary, policy and model versions
Scalability	Persisted features, Parquet snapshots, bounded evaluation, asynchronous workflows, pagination
Integrity	Closed business dates, chronological cutoffs, checksums, immutable model bundles, duplicate protection
Availability	Auditable fallback when the ML service is unavailable; case creation occurs after commit

Quality	Requirement and implementation direction
Maintainability	Separate Angular, Spring Boot, and FastAPI applications with stable REST contracts

3.3 Overall Architecture

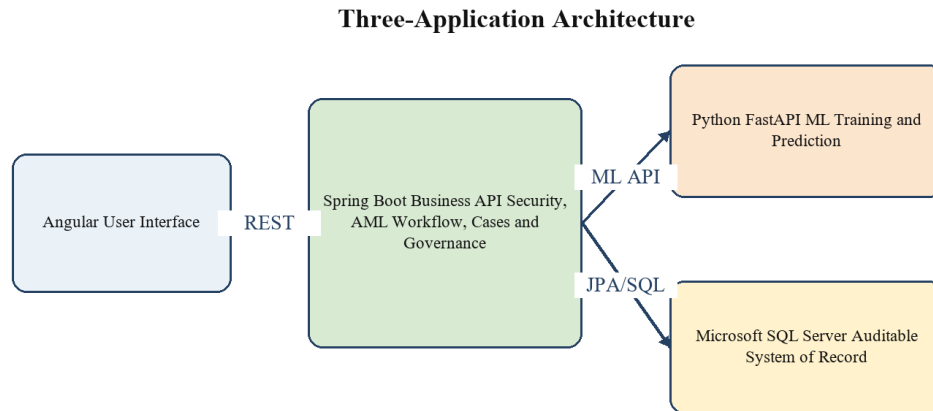


Figure 1. Three-application architecture

Angular provides the browser interface. Spring Boot owns transaction persistence, security, feature orchestration, risk policy, model governance, and case workflow. FastAPI owns numerical preprocessing, model fitting, artifact loading, prediction, and research analysis. SQL Server is the auditable system of record. This division prevents the Python service from directly mutating business records and prevents the UI from accessing model artifacts or database tables.

3.4 Database and Audit Design

Core operational records include transactions, upload batches, persisted transaction features, learning eligibility, prediction logs, fraud alerts, cases, case notes, STR reports, application configuration, training runs, immutable model registry entries, validation reports, deployment pointers, and growth studies. Migration scripts are additive and independently verifiable. Training runs reference closed business windows and immutable exported datasets rather than mutable lifetime queries.

3.5 Transaction Decision Flow

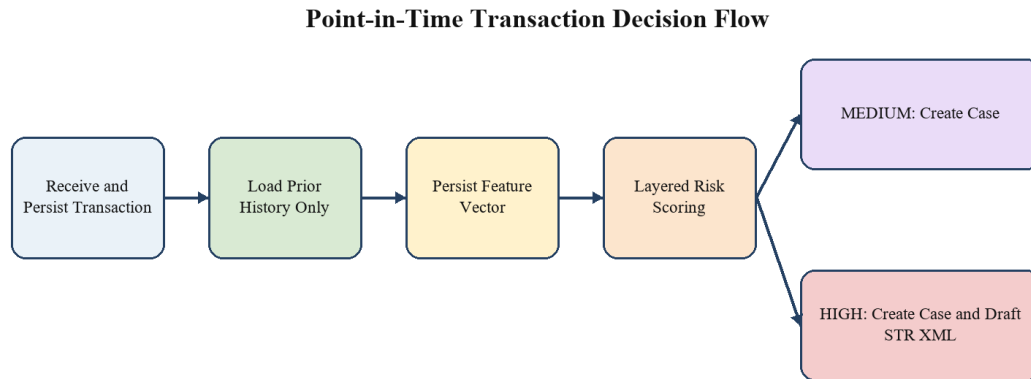


Figure 2. Point-in-time transaction decision flow

A transaction is first persisted with processing status. Feature engineering loads only records earlier than the transaction timestamp, preventing future information leakage. The feature vector is persisted before prediction. The layered orchestrator returns the final risk score, model evidence, feature summary, reasons, and action. Suspicious results create an alert and publish an after-commit event. MEDIUM creates a case; HIGH creates a case and automatically generates draft STR XML asynchronously so the transaction database commit is not held open by case work.

3.6 Authentication and Authorization

Login issues a signed JWT. Passwords are stored using BCrypt with strength 12. Spring Security permits health, login, registration, and API documentation without authentication, requires authentication for all other APIs, and restricts upload, training, AML writes, and comparison writes to ADMIN or AML_ADMIN. Angular route guards improve usability, but the backend remains the authority.

CHAPTER 4

METHODOLOGY

4.1 Global Learning Mode

The Settings page stores `system.learning_mode` in `app_config`. UNSUPERVISED is selected when confirmed labels are absent. SUPERVISED is selected only when an auditable FraudLabel is present. Upload validation, the model catalog, Model Tuning, Training Operations, the comparison page, and the Risk Policy model list read this mode so the workflow remains consistent.

Table 3. Dual learning model catalog

Mode	Models	Required data	Primary interpretation
Unsupervised	Isolation Forest; Autoencoder; Local Outlier Factor	Transactions without confirmed class labels	Anomaly evidence and stability
Supervised	XGBoost; Random Forest; Logistic Regression	Reviewed binary FraudLabel	Estimated suspicious probability and classification quality

4.2 Feature Engineering and Leakage Prevention

Features describe amount and balance ratios, hour and day, night and weekend indicators, customer history count, rolling amount statistics, z-scores, debit and credit ratios, transaction velocity, unique beneficiaries, new location/channel/device indicators, peer-group statistics, expected turnover deviation, customer type, and risk rating. Each feature is computed using records available before the prediction time. The feature schema and column order are persisted with every model bundle so training and prediction cannot silently disagree.

4.3 Customer Behaviour Scoring

The customer layer asks whether the transaction is unusual for the same customer. A transparent weighted formula combines amount deviation, transaction frequency, time-gap deviation, categorical novelty, and unusual hour. A confidence multiplier reduces

customer-specific evidence during cold start. This prevents two or three prior transactions from being treated as a mature profile while retaining limited evidence.

4.4 Peer Behaviour Scoring

The peer layer compares the transaction with similar customers. Groups are derived from customer attributes such as occupation and age band, not hard-coded for one profession. If a specific occupation-age baseline lacks enough history, the scorer falls back to a broader parent segment and then to a global baseline, with decreasing confidence. The peer window is anchored to the training snapshot cutoff rather than the newest just-arrived transaction, preventing one future transaction from shifting the historical reference window.

4.5 Deterministic AML Rules

Rules evaluate transparent conditions such as amount multiples, customer and peer z-scores, high transaction velocity, repeated near-threshold activity, expected-turnover deviation, unusual locations, and hard compliance overrides. Rule thresholds are configured separately from model tuning. A hard override remains identifiable as a rule decision rather than being misrepresented as model consensus.

4.6 Layered Weighted Risk Aggregation

Layered Risk Scoring (All Weights Normalize to 100%)

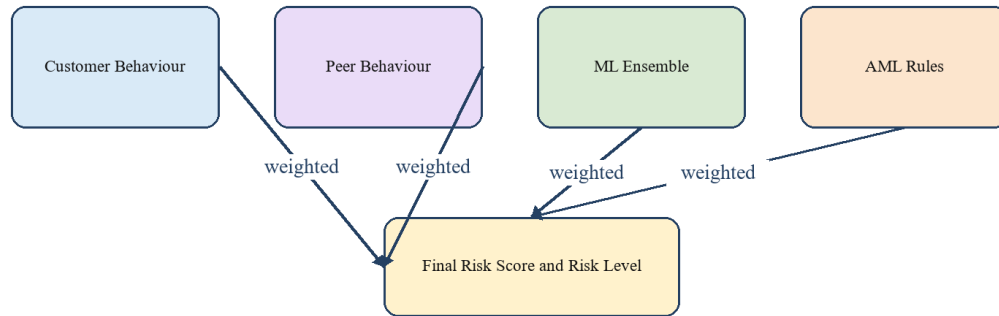


Figure 3. Layered risk scoring

The final normalized score is $\text{customerWeight} \times \text{customerScore} + \text{peerWeight} \times \text{peerScore} + \text{mlWeight} \times \text{mlScore} + \text{rulesWeight} \times \text{rulesScore}$. Top-level weights total 1.0. Inside the ML ensemble, enabled model weights also total 1.0. The UI provides component-level normalization so an operator can increase one value to use the remaining percentage without unpredictably changing unrelated sections. Risk thresholds convert the final 0-1 value to a 0-100 display and NORMAL, LOW, MEDIUM, or HIGH.

4.7 Chronological Growth Comparison

The research experiment orders the immutable snapshot by `transaction_date` and trains each selected model on the oldest 10%, 25%, 50%, and 100% that meet the minimum-row rule. Each partition uses a chronological holdout, so future rows are not randomly mixed into training. This directly answers how behaviour, stability, anomaly rate, training time, and throughput change as historical evidence increases.

Table 4. Evaluation metrics by learning mode

Mode	Metrics	What they establish
Unsupervised	EM-AUC, score separation, skewness, anomaly rate, rate control, stability, throughput	Label-free diagnostic quality; not fraud accuracy

Mode	Metrics	What they establish
Supervised	PR-AUC, ROC-AUC, precision, recall, F1, Brier score, confusion matrix	Classification and probability quality on reviewed labels
Both	Training duration, learned rows, evaluation rows, prediction latency	Operational scalability and reproducibility

4.8 Governed Training

Governed Training and Comparison Pipeline

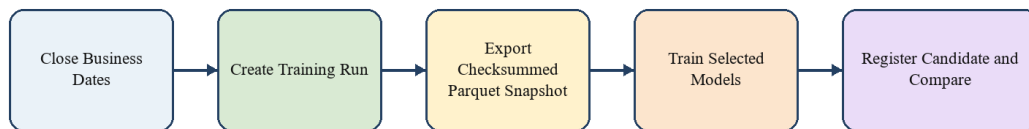


Figure 4. Governed training and comparison pipeline

An administrator chooses when to train; automatic daily and weekly schedules were removed. Business dates must first be closed. Spring Boot exports eligible persisted features through keyset pagination into checksummed Parquet parts. Python trains only the models selected from the active mode and saves versioned artifacts. Spring registers candidates with metrics and metadata. Training does not silently activate a model.

CHAPTER 5

IMPLEMENTATION

5.1 Tools and Technologies

Table 5. Main implementation technologies

Layer	Technology	Purpose
Frontend	Angular 21, TypeScript, RxJS	Responsive pages, forms, route guards, API integration, alerts
Business backend	Java 17, Spring Boot 4, Spring Security, JPA	Transactions, security, policies, cases, training orchestration
ML service	Python, FastAPI, pandas, NumPy, scikit-learn, XGBoost	Training, prediction, evaluation, artifact handling
Database	Microsoft SQL Server	Auditable operational and governance state
Data interchange	REST/JSON, multipart upload, Parquet, XML	Online calls, datasets, and STR draft output

5.2 Angular User Interface

The UI uses a white, responsive dashboard with a toggleable side menu and consistent success/error popup alerts. Current routes include Uploads, Datasets and model comparison, Supervised Comparison, Training Operations, Risk Policy, Settings, Model Tuning, Transaction Check, Case Management, and Manual Case Creation. Administrative pages are route-protected, and every page delegates authority to backend APIs.

5.3 Risk Policy Configuration

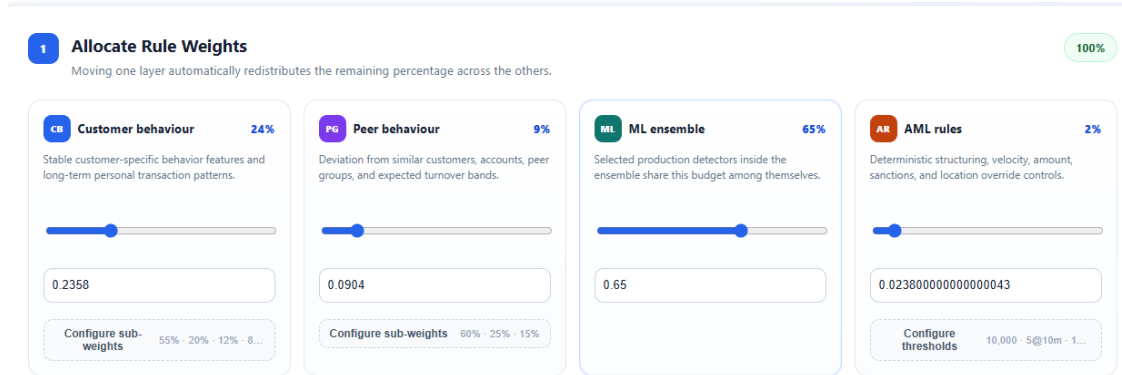


Figure 5. Risk policy configuration interface

The configuration interface allocates 100% across customer behaviour, peer behaviour, the ML ensemble, and AML rules. A modal selects active production models based on the global learning mode. Disabled models do not participate. The page displays effective model contribution to the total score and prevents totals above 100%.

5.4 Model Tuning

Model Tuning is mode-aware. In unsupervised mode it exposes controls for Isolation Forest, Autoencoder, and LOF. In supervised mode it exposes XGBoost tree count, depth, learning rate, row and feature sampling; Random Forest tree count, depth, and leaf size; and Logistic Regression regularization and iteration limits. Shared evaluation controls include minimum rows, holdout fraction, maximum evaluation rows, and random seed. Values are persisted in `app_config` and consumed by both training and growth analysis.

5.5 Transaction Check and Explainability

Transaction details

Reset

Transaction ID	Account ID
TXN-1786436891687	AC00455
Amount	Account balance
2000	Optional
Transaction type	Channel
TRANSFER	ATM
Transaction date	Location
08/11/2026 08:28 AM	Mesa
Customer age optional	Customer occupation optional
68	Doctor
Login attempts	
2	

Check Transaction

Figure 6. Transaction entry interface

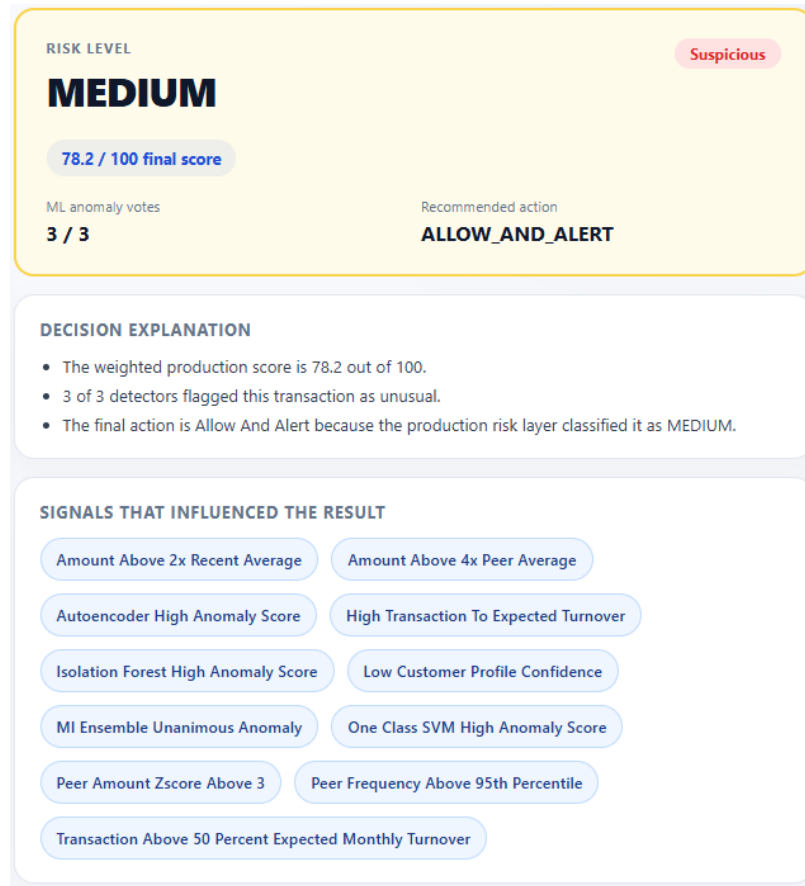


Figure 7. Explainable transaction decision

The transaction page accepts a transaction ID, account, amount, balance, type, channel, date, location, age, occupation, and login attempts. The result displays final score, risk level, action, ML anomaly votes, human-readable reason tags, key features, model diagnostics, latency, layered component scores, policy version, and raw payload for technical review. This separation prevents the raw model decision from being confused with the final policy decision.

5.6 Case Management and STR

Suspicious transactions create fraud alerts and cases after the transaction commit. Case lists use server-side pagination, filtering, and summary counts to avoid loading the full case table. The detail view includes transaction evidence, prediction evidence, analyst notes, reason explanations, and allowed actions. A reviewer can mark a case false positive or generate draft STR XML, but the state machine prevents STR generation from a false-

positive case and prevents an STR-generated case from later being marked false positive. Manual cases reuse existing transactions and avoid duplicates.

5.7 Training and Artifact Management

Training operations expose model checkboxes from the active Risk Policy rather than fixed daily or weekly training types. One run can train selected candidates from the same immutable snapshot. Artifacts include the fitted estimator, scaler or preprocessing state, ordered feature columns, threshold, hyperparameters, metrics, and dataset checksum. Atomic replacement avoids partially written model directories, and model loading caches versioned artifacts for prediction.

5.8 Failure Handling

If FastAPI is unavailable, Spring records the failure and returns the documented safe fallback instead of corrupting transaction state. Invalid roles receive JSON 401 or 403 responses. Duplicate transaction IDs are rejected. Training failures preserve messages and do not activate candidates. Dataset export rejects open business dates. Artifact checksum and feature-version mismatches block governance actions.

CHAPTER 6

TESTING AND RESULTS

6.1 Testing Strategy

Testing combined automated unit and integration tests with manual end-to-end acceptance checks. Java tests cover feature context, customer and peer scoring, rules, layered risk arithmetic, training export, model registry, validation, deployment, case management, authentication, and comparison services. Python tests cover growth analysis, persisted-feature alignment, artifact bundles, route compatibility, finite-value checks, and model artifact reuse. Angular production builds verify template and TypeScript integration.

6.2 Current Unsupervised Training Result

Current models Analysis			
METRIC	ISOLATION FOREST UNSUPERVISED	AUTOENCODER UNSUPERVISED	LOCAL OUTLIER FACTOR UNSUPERVISED
Learned rows	50,000	50,000	50,000
EM-AUC COMPARABLE	67.2 / 100	85.6 / 100	89.2 / 100
Score skewness COMPARABLE	77.5 / 100	100.0 / 100	100.0 / 100
Anomaly rate	1.00%	2.50%	0.82%
Average score NATIVE SCALE	-0.057	-0.357	-0.490
Score P95 NATIVE SCALE	-0.021	-0.137	-0.266
Score P99 NATIVE SCALE	0.000	0.401	-0.033
Alert count	500	1,250	412
Validation rows	50,000	50,000	50,000
Training duration	1,309 ms	3,194 ms	5,595 ms
Registered at	Aug 12, 2026, 9:58:52 AM	Aug 12, 2026, 9:58:52 AM	Aug 12, 2026, 9:58:53 AM
Feature version	AML_FEATURES_V2	AML_FEATURES_V2	AML_FEATURES_V2

Figure 8. Model comparison results for the 50,000-row training snapshot

Table 6. Current unsupervised model results on 50,000 rows

Model	Rows	EM-AUC	Skewness	Anomaly rate	Training time
Isolation Forest	50,000	67.2	77.5	1.00%	1,309 ms
Autoencoder	50,000	85.6	100.0	2.50%	3,194 ms
Local Outlier Factor	50,000	89.2	100.0	0.82%	5,595 ms

Local Outlier Factor produced the strongest current EM-AUC score, while Autoencoder also showed strong label-free separation. Isolation Forest trained fastest but had a lower EM-AUC. These figures describe diagnostic separation and anomaly behaviour on the available snapshot; they do not prove fraud accuracy because the underlying unsupervised dataset has no confirmed fraud label. Selection must also consider growth stability, throughput, alert volume, and operational latency.

6.3 Explainable Transaction Scenario

In a tested transaction scenario, the three selected ML detectors all flagged an anomaly. The final score nevertheless depended on the configured 65% ML allocation plus customer, peer, and rule contributions. Peer scoring identified the amount as above the peer average and the frequency above the 95th percentile; the rule layer detected turnover and amount deviations. The result reached MEDIUM and produced ALLOW_AND_ALERT with a case. This demonstrates why model votes and final risk must be displayed separately.

6.4 Automated Validation Summary

Table 7. Automated validation summary

Validation	Observed result
Spring Boot compilation	319 Java source files compiled successfully
Python automated suite	10 tests passed in the latest focused ML validation run
Angular production build	Bundle generated successfully; only existing size-budget warnings
Security design	JWT filter, BCrypt, authenticated APIs, ADMIN/AML_ADMIN write restrictions
Case safeguards	False-positive/STR state conflicts rejected; pagination and summary queries implemented
Research integrity	Oldest-first partitions, chronological holdout, immutable dataset checksum

6.5 Performance and Scalability Observations

The training screenshot shows all three unsupervised models completing on 50,000 rows within approximately 1.3 to 5.6 seconds on the development environment. This is not

a bank-scale benchmark, but it validates the pipeline and exposes relative cost. Real-time latency is reduced by reusing loaded artifacts, persisting feature vectors, limiting historical reads to point-in-time windows, and moving case/STR work to an asynchronous after-commit listener. Larger deployments should benchmark p50, p95, and p99 latency under concurrent load.

6.6 Scientific Interpretation Limits

EM-AUC, separation, score skewness, anomaly-rate control, Silhouette score, Davies-Bouldin index, and Calinski-Harabasz index are not fraud accuracy. The first group describes anomaly score structure; the clustering indices describe geometry only when a genuine clustering experiment is performed. Precision, recall, F1, and false-positive rate require reviewed outcomes. The UI and report explicitly preserve this distinction.

CHAPTER 7

PROBLEMS AND SOLUTIONS

7.1 Problem-to-Solution Summary

Table 8. Problem-to-solution matrix

Problem	Implemented solution	Result
No reliable fraud labels	Global unsupervised mode with Isolation Forest, Autoencoder, and LOF; label-free metrics	Useful anomaly research without false accuracy claims
Some clients have reviewed labels	Separate supervised mode with XGBoost, Random Forest, Logistic Regression and supervised metrics	One platform supports both scientifically valid workflows
Model behaviour changes with data volume	Oldest-first 10%, 25%, 50%, 100% growth study with chronological holdout	Stability and scalability become visible before selection
Raw scores are incompatible	Normalize every model and component to 0-1 before weighted aggregation	All configured contributions total exactly 100%
ML alone misses AML obligations	Separate deterministic AML rules with hard override support	Mandatory controls remain transparent and auditable
One customer differs from population	Customer behaviour scorer with amount, velocity, novelty, time-gap, and confidence	Personal deviations contribute independently
Peer comparison unavailable for sparse groups	Occupation-age group, parent segment, then global fallback with confidence reduction	Every transaction can receive a bounded peer result
Cold start creates unstable profiles	Minimum history gate and confidence multiplier; suspicious rows excluded from trusted learning	Immature or contaminated profiles have less authority
Training and prediction feature drift	Persisted versioned feature vectors and exact feature-column artifact	Training and scoring use the same schema
Millions of rows cannot be sent as JSON	Keyset export, Parquet parts, checksums, capped evaluation, selected-model training	Memory and network usage remain bounded
Transaction checks were slow	Cached model artifacts, persisted features, bounded history, asynchronous post-commit case work	Decision path avoids retraining and case I/O
Model votes conflicted with final score	Display votes as diagnostics and calculate final score from explicit policy weights	Users can explain why 3/3 anomalies may still be below threshold
Weight sliders changed unrelated values	Normalize within each component and model section; enforce totals server-side	Policy editing is predictable and cannot exceed 100%
Cases became expensive to list	Server-side pagination, filtering, count summaries, and reduced N+1 loading	Case management scales beyond small demonstrations

Problem	Implemented solution	Result
Alerts did not complete the AML workflow	Automatic cases, notes, false-positive outcome, manual cases, and draft STR XML	Suspicious decisions become reviewable operational records
Administrative APIs were exposed	JWT authentication, BCrypt, role authorization, deny-by-default security	Sensitive operations require authorized identities
Training could silently change production	Immutable model registry, validation, version locking, controlled promotion, rollback	Model lifecycle is auditable and reversible

7.2 How Scalability Is Solved

Scalability is addressed by separating the online decision path from offline training. Online scoring reads bounded point-in-time state and already loaded artifacts. Offline training uses closed windows and immutable Parquet snapshots. The UI sends a training-run identifier rather than millions of feature rows. Evaluation rows are capped, large algorithms can use bounded samples, and candidate creation is manual and asynchronous. SQL Server remains the governance store rather than the numerical training engine.

7.3 Kafka Integration Roadmap

Kafka is not required for the current prototype, but it is the next transport layer for millions of daily events. A SQL outbox should publish committed transaction events to account-keyed topics. Independent consumers can update customer, peer, and velocity state; write feature events; request predictions; persist model results; and create cases. Account-keyed partitioning preserves ordering for one customer while consumer groups provide horizontal scaling, replay, and back-pressure isolation. Kafka transports events; it does not replace SQL governance or train models by itself.

7.4 Why the Final Design Is More Flexible

The client decides the learning methodology once at system level, chooses participating production models based on comparison evidence, assigns model and layer weights, and manually starts training when data is ready. The same business layers continue to operate in both modes. This avoids forcing a bank with no labels into supervised metrics and avoids preventing a bank with reviewed labels from using probability-based classifiers.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 Conclusion

The project delivers a complete prototype for transaction ingestion, feature persistence, dual-mode machine learning, model growth comparison, explainable layered risk scoring, governed training, authentication, and AML case management. Its most important design decision is methodological honesty: anomalies are treated as suspicious evidence, not confirmed fraud, unless reviewed labels justify supervised evaluation.

The system solves the central project problems by using point-in-time features, immutable chronological datasets, mode-specific models and metrics, normalized risk weights, transparent customer and peer scorers, deterministic rules, auditable model versions, and a reviewer workflow. The Angular interface makes these decisions visible rather than hiding them behind one prediction label.

8.2 Current Limitations

- The 50,000-row result is a development-scale experiment, not a production bank benchmark.
- Unsupervised results cannot provide true precision, recall, F1, or confirmed false-positive rate.
- The generated supervised dataset is synthetic and must not replace reviewed institutional labels for real deployment.
- Kafka, distributed feature state, centralized observability, and infrastructure deployment are architectural roadmaps rather than completed production services.
- Draft STR XML requires validation against the exact schema and submission process of the relevant regulator.
- Model calibration, fairness by segment, privacy controls, penetration testing, and disaster recovery require further work.

8.3 Future Work

- Integrate Kafka through a transactional SQL outbox and account-keyed consumer groups.
- Add online feature stores or bounded materialized state for customer and peer statistics.
- Collect reviewer labels and add calibration curves, threshold optimization, recall at fixed false-positive rates, and drift by business period.
- Run reproducible load tests with millions of rows and concurrent transaction scoring, reporting p50, p95, and p99 latency.
- Introduce approved clustering experiments using MiniBatchKMeans or BIRCH and display PCA/UMAP projections with Silhouette, Davies-Bouldin, and Calinski-Harabasz as geometric diagnostics.
- Add containerization, centralized logs and metrics, secrets management, backup, and automated deployment pipelines.
- Validate STR XML and workflow against the applicable jurisdiction and add digital approval controls.

8.4 Final Remarks

The completed platform demonstrates that a strong fraud and AML project is not only a model-selection exercise. It is a data-integrity, scalability, explainability, governance, and human-review problem. By making each of those concerns explicit, the project provides a credible foundation for further academic study and future production engineering.

REFERENCES

- [1] F. T. Liu, K. M. Ting, and Z.-H. Zhou, 'Isolation Forest,' Proceedings of the 8th IEEE International Conference on Data Mining, 2008.
- [2] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, 'LOF: Identifying Density-Based Local Outliers,' Proceedings of ACM SIGMOD, 2000.
- [3] T. Chen and C. Guestrin, 'XGBoost: A Scalable Tree Boosting System,' Proceedings of the 22nd ACM SIGKDD Conference, 2016.
- [4] L. Breiman, 'Random Forests,' Machine Learning, vol. 45, pp. 5-32, 2001.
- [5] D. P. Kingma and M. Welling, 'Auto-Encoding Variational Bayes,' International Conference on Learning Representations, 2014.
- [6] Spring, 'Spring Boot Reference Documentation' and 'Spring Security Reference,' accessed August 2026.
- [7] Angular, 'Angular Documentation,' accessed August 2026.
- [8] FastAPI, 'FastAPI Documentation,' accessed August 2026.
- [9] scikit-learn developers, 'scikit-learn User Guide,' accessed August 2026.
- [10] Apache Software Foundation, 'Apache Parquet Documentation' and 'Apache Kafka Documentation,' accessed August 2026.
- [11] Microsoft, 'SQL Server Documentation,' accessed August 2026.

APPENDIX A: MAIN USER WORKFLOW

1. Log in with an authorized account.
2. Open Settings and choose supervised or unsupervised learning.
3. Upload CSV/Excel data and inspect the batch result.
4. Close the relevant business dates in Training Operations.
5. Create a training run, generate the immutable dataset, and wait for DATASET_READY.
6. Choose the desired active-mode models and start training.
7. Open the corresponding comparison page and review quality, stability, anomaly/classification metrics, and runtime.
8. Open Risk Policy, select production models, and configure normalized layer/model weights and thresholds.
9. Use Transaction Check or the transaction API to score new activity.
10. Review automatically generated cases; mark false positives or generate/download STR XML as appropriate.

APPENDIX B: SELECTED API ENDPOINTS

Table B1. Selected API endpoints

Method	Endpoint	Purpose
POST	/api/auth/login	Authenticate and obtain JWT
POST	/api/v1/uploads/transactions	Upload CSV or Excel transactions
POST	/api/v1/transactions	Persist and score one transaction
POST	/api/v1/aml/training-runs/pipeline/start	Create governed training run
POST	/api/v1/aml/training-runs/{id}/dataset	Generate immutable feature dataset
POST	/api/v1/aml/growth-analysis/training-runs/{id}	Run chronological growth analysis
GET/PUT	/api/v1/anomaly-model-comparisons/risk-policy	Read or save production risk policy
GET/PUT	/api/v1/anomaly-model-comparisons/model-tuning	Read or save active-mode tuning
GET	/api/cases	Paginated case search
POST	/api/cases/{id}/false-positive	Record false-positive outcome
POST	/api/cases/{id}/str-xml	Generate draft STR XML

APPENDIX C: CONFIGURATION PRINCIPLES

- Top-level risk-layer weights must total 100%.
- Enabled model weights inside the ML ensemble must total 100%.
- Supervised mode requires reviewed labels; unresolved labels are excluded from training.
- Unsupervised metrics must never be described as confirmed fraud accuracy.
- Training uses closed business dates and a fixed cutoff.
- No training run automatically activates or promotes a model.
- MEDIUM creates a case; HIGH creates a case and draft STR XML under the current policy.
- The backend is the security authority even when UI controls are hidden or disabled.